

Context

– это интерфейс, который несет в себе:

1. Сигнал отмены (deadline/timeout или явная отмена)
2. Значения (ключ-значение) для передачи сквозь вызовы

Его главная цель – управление жизненным циклом операций в конкурентной среде (горutiнах).

```
type Context interface {
    Deadline() (deadline time.Time, ok bool) // время отмены
    Done() <-chan struct{}                  // канал-сигнал отмены
    Err() error                             // причина отмены
    Value(key interface{}) interface{}      // получение значения
}
```

Зачем?

Задача	Описание
Отмена операций	Остановить долгие процессы (запросы, вычисления)
Таймауты	Ограничить время выполнения
Graceful shutdown	Плавно завершить приложение
Сквозные данные	Передать traceID, userID через все слои
Предотвращение утечек	Остановить “зависшие” горутины

Корневой контекст

– это начало цепочки

```
context.Background() // для main, обработчиков запросов, тестов
context.TODO()       // когда не знаете, какой использовать (заглушка)
```

Методы для создания производственных контекстов

context.WithCancel(parent) – ручная отмена

Используем, когда нужно самим решить, когда остановить процесс

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // Создаем контекст с ручной отменой
    ctx, cancel := context.WithCancel(context.Background())

    // ВАЖНО: всегда вызываем cancel при выходе
    defer cancel()

    // Запускаем горутину, которая работает
    go func() {
        for {
            select {
            case <-ctx.Done():
```

```

        // Получили сигнал остановки
        fmt.Println("Горутина остановлена")
        return
    default:
        // Имитируем работу
        fmt.Println("Работаю...")
        time.Sleep(500 * time.Millisecond)
    }
}

// Даем поработать 2 секунды
time.Sleep(2 * time.Second)

// Останавливаем горутину
fmt.Println("Отменяем контекст")
cancel()

// Даем время на завершение
time.Sleep(500 * time.Millisecond)
}

```

Результат:

```

Работаю...
Работаю...
Работаю...
Работаю...
Отменяем контекст
Горутина остановлена

```

context.WithTimeout(parent, duration) – отмена через время

Используем, чтобы ограничить временем, например, для запросов к БД, API, внешним сервисам

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // Создаем контекст, который отменится через 3 секунды
    ctx, cancel := context.WithTimeout(context.Background(), 3*time.Second)
    defer cancel() // ВСЕГДА вызываем!

    fmt.Println("Начинаем работу, таймаут 3 секунды")

    // Имитируем долгую работу
    select {
    case <-ctx.Done():
        // Контекст отменился (по таймауту)
        fmt.Println("Время вышло:", ctx.Err())
        // ctx.Err() вернет "context deadline exceeded"
    case <-time.After(5 * time.Second):
        // Это не выполнится, т.к. таймаут наступит раньше
        fmt.Println("Работа завершена успешно")
    }
}
```

Результат:

Начинаем работу, таймаут 3 секунды
Время вышло: context deadline exceeded
context.WithDeadline() – остановка в конкретное время

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // Устанавливаем дедлайн через 5 секунд от текущего времени
    deadline := time.Now().Add(5 * time.Second)
    ctx, cancel := context.WithDeadline(context.Background(), deadline)
    defer cancel()

    fmt.Println("Дедлайн в:", deadline.Format("15:04:05"))
    fmt.Println("Текущее время:", time.Now().Format("15:04:05"))

    // Ждем 6 секунд (дольше дедлайна)
    time.Sleep(6 * time.Second)
}
```

```

// Проверяем, отменился ли контекст
if ctx.Err() != nil {
    fmt.Println("Контекст отменен:", ctx.Err())
}
}

```

Результат:

Дедлайн в: 14:25:30
 Текущее время: 14:25:25
 Контекст отменен: context deadline exceeded
 context.WithValue() – передача данных

```

package main

import (
    "context"
    "fmt"
)

// ВАЖНО: создаем свой тип для ключа, чтобы избежать конфликтов
type myKey string

func main() {
    // Определяем ключи
    const userIDKey myKey = "userID"
    const traceIDKey myKey = "traceID"

    // Создаем контекст с данными
    ctx := context.Background()
    ctx = context.WithValue(ctx, userIDKey, 12345)
    ctx = context.WithValue(ctx, traceIDKey, "abc-123-xyz")

    // Передаем в функцию
    processRequest(ctx)
}

func processRequest(ctx context.Context) {
    // Извлекаем данные из контекста
    userID := ctx.Value(myKey("userID"))
    traceID := ctx.Value(myKey("traceID"))

    // Проверяем, что данные есть
    if userID != nil && traceID != nil {
        fmt.Printf("Обработка запроса: UserID=%v, TraceID=%v\n", userID, traceID)
    } else {
        fmt.Println("Данные не найдены")
    }
}

```

Результат:

Обработка запроса: UserID=12345, TraceID=abc-123-xyz
 Важно: ключи должны быть уникальными для каждого пакета, чтобы не было конфликтов.

5. КАК ПРОВЕРИТЬ ОТМЕНУ (3 способа)

Способ 1: Через select (ждем сигнал)

```
func doWork(ctx context.Context) {
    select {
    case <-ctx.Done():
        fmt.Println("Остановлен:", ctx.Err())
        return
    case <-time.After(10 * time.Second):
        fmt.Println("Работа завершена")
    }
}
```

Способ 2: Через ctx.Err() (не ждем)

```
func doWork(ctx context.Context) {
    if ctx.Err() != nil {
        fmt.Println("Уже отменен:", ctx.Err())
        return
    }
    // продолжаем работу
}
```

Способ 3: В цикле (постоянно проверяем)

```
func processItems(ctx context.Context, count int) {
    for i := 0; i < count; i++ {
        // Проверяем на каждой итерации
        select {
        case <-ctx.Done():
            fmt.Println("Прервано на итерации", i)
            return
        default:
            // Делаем работу
            fmt.Println("Обработано:", i)
            time.Sleep(500 * time.Millisecond)
        }
    }
}
```

6. КОМБИНИРОВАННЫЙ ПРИМЕР (все вместе)

```
package main

import (
    "context"
    "fmt"
    "time"
)

type key string
const requestIDKey key = "requestID"

func main() {
    // 1. Создаем корневой контекст
    ctx := context.Background()

    // 2. Добавляем данные
    ctx = context.WithValue(ctx, requestIDKey, "REQ-001")

    // 3. Добавляем таймаут 3 секунды
    ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
```

```

    defer cancel()

    // 4. Запускаем обработку
    result := process(ctx)
    fmt.Println("Результат:", result)
}

func process(ctx context.Context) string {
    // Получаем ID запроса из контекста
    reqID := ctx.Value(requestIDKey)
    fmt.Println("Обработка запроса:", reqID)

    // Имитируем работу с проверкой отмены
    for i := 1; i <= 10; i++ {
        select {
        case <-ctx.Done():
            return "Остановлен: " + ctx.Err().Error()
        default:
            fmt.Println("Шаг", i)
            time.Sleep(500 * time.Millisecond)
        }
    }

    return "Успешно завершено"
}

```

Результат:

Обработка запроса: REQ-001

Шаг 1

Шаг 2

Шаг 3

Шаг 4

Шаг 5

Результат: Остановлен: context deadline exceeded

7. САМЫЙ ПРОСТОЙ ПРИМЕР ДЛЯ ПОНИМАНИЯ

```
package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // Создаем контекст, который отменится через 2 секунды
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel()

    // Запускаем горутину
    go func() {
        // Каждые 500ms печатаем сообщение
        for {
            select {
            case <-ctx.Done():
                // Контекст отменен - выходим
                fmt.Println("Выходим из горутины")
                return
            default:
                fmt.Println("Еще работаем")
                time.Sleep(500 * time.Millisecond)
            }
        }
    }()

    // Ждем 3 секунды (горутина остановится через 2 сек)
    time.Sleep(3 * time.Second)
    fmt.Println("Программа завершена")
}
```

Результат:

```
Еще работаем
Еще работаем
Еще работаем
Еще работаем
Выходим из горутины
Еще работаем    // это напечатает main перед завершением
Программа завершена
```

8. ПРАВИЛА (ЗАПОМНИТЬ!)

```
// ПРАВИЛЬНО: контекст первым аргументом
func GetUser(ctx context.Context, id int) string

// НЕПРАВИЛЬНО: контекст не первым
func GetUser(id int, ctx context.Context) string

// ПРАВИЛЬНО: всегда defer cancel()
ctx, cancel := context.WithTimeout(parent, time.Second)
defer cancel()

// НЕПРАВИЛЬНО: забыли cancel()
ctx, _ := context.WithTimeout(parent, time.Second)

// ПРАВИЛЬНО: типизированный ключ
type key string
const myKey key = "myKey"

// НЕПРАВИЛЬНО: строковый ключ
ctx = context.WithValue(ctx, "key", value)

// ПРАВИЛЬНО: не храним контекст в структуре
type Service struct {
    // нет поля ctx
}
func (s *Service) Do(ctx context.Context) {}

// НЕПРАВИЛЬНО: храним контекст
type Service struct {
    ctx context.Context
}
```


9. ТАБЛИЦА: КОГДА ЧТО ИСПОЛЬЗОВАТЬ

Ситуация	Используй
Главная функция main()	context.Background()
Тесты	context.Background()
Временно не знаешь контекст	context.TODO()
Нужно остановить вручную	context.WithCancel()
Ограничить время выполнения	context.WithTimeout()
Закончить к определенному времени	context.WithDeadline()
Передать данные (traceID, userID)	context.WithValue()
HTTP-обработчик	r.Context()
Запрос в БД	context.WithTimeout()

10. ПРОВЕРКА СЕБЯ (вопросы)

Вопрос 1: Что будет, если не вызвать cancel()? Ответ: Контекст и все связанные с ним ресурсы не освободятся → утечка.

Вопрос 2: Можно ли отменить дочерний контекст, не отменяя родительский? Ответ: Да, у каждого дочернего контекста своя функция cancel().

Вопрос 3: Что вернет ctx.Err() для активного контекста? Ответ: nil.

Вопрос 4: Что будет, если вызвать cancel() дважды? Ответ: Ничего, второй вызов не имеет эффекта.

11. БЫСТРАЯ ШПАРГАЛКА

```
// 1. Создать контекст
ctx := context.Background()
ctx := context.TODO()

// 2. С отменой
ctx, cancel := context.WithCancel(ctx)
defer cancel()

// 3. С таймаутом
ctx, cancel := context.WithTimeout(ctx, 5*time.Second)
defer cancel()

// 4. С дедлайном
ctx, cancel := context.WithDeadline(ctx, time.Now().Add(5*time.Second))
defer cancel()

// 5. С данными
type key string
const myKey key = "myKey"
ctx = context.WithValue(ctx, myKey, "value")

// 6. Получить данные
val := ctx.Value(myKey)

// 7. Проверить отмену
select {
case <-ctx.Done():
    // отменен
default:
    // работает
}

// 8. Проверить ошибку
if ctx.Err() != nil {
    // отменен
}
```

context и сигналы

Сигналы приходят от ОС и позволяют приложению реагировать на внешние события:

Сигнал	Когда приходят	Что обычно делают
SIGINT	Ctrl+C	Остановить приложение
SIGTERM	kill	Остановить приложение
SIGKILL	kill -9	Мгновенное убийство
SIGHUP	Перезагрузка терминала	Перечитать конфиг
SIGUSR1/SIGUSR2	Пользовательские сигналы	Кастомные действия

Пример:

```
package main

import (
    "fmt"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    // Создаем канал для сигналов
    sigChan := make(chan os.Signal, 1)

    // Подписываемся на сигналы
    signal.Notify(sigChan, syscall.SIGINT, syscall.SIGTERM)

    fmt.Println("Приложение запущено. Нажмите Ctrl+C для остановки.")

    // Блокируемся, пока не придет сигнал
    sig := <-sigChan
    fmt.Printf("Получен сигнал: %v\n", sig)
    fmt.Println("Завершаем работу...")
}
```

Но главная красота в другом:

Сигналы + Контекст = Graceful Shutdown

```
package main

import (
    "context"
    "fmt"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    // 1. Создаем контекст, который отменится при сигнале
    ctx, cancel := context.WithCancel(context.Background())
```

```

defer cancel()

// 2. Ловим сигналы
sigChan := make(chan os.Signal, 1)
signal.Notify(sigChan, syscall.SIGINT, syscall.SIGTERM)

// 3. Запускаем горютины, которые слушают контекст
go worker(ctx, "Воркер-1")
go worker(ctx, "Воркер-2")

// 4. Ждем сигнал
sig := <-sigChan
fmt.Printf("\nПолучен сигнал: %v\n", sig)
fmt.Println("Начинаем graceful shutdown...")

// 5. Отменяем контекст - все горютины получают сигнал остановки
cancel()

// 6. Даем время на завершение (важно!)
time.Sleep(2 * time.Second)
fmt.Println("Приложение завершено")
}

func worker(ctx context.Context, name string) {
    for {
        select {
            case <-ctx.Done():
                fmt.Printf("%s: остановлен\n", name)
                return
            default:
                fmt.Printf("%s: работаю...\n", name)
                time.Sleep(500 * time.Millisecond)
        }
    }
}

```

С таймаутом

```

package main

import (
    "context"
    "fmt"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    // Основной контекст для работы
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()

    // Канал для сигналов
    sigChan := make(chan os.Signal, 1)
    signal.Notify(sigChan, syscall.SIGINT, syscall.SIGTERM)
}

```

```

// Запускаем воркеров
go worker(ctx, "Воркер-1")
go worker(ctx, "Воркер-2")

// Ждем сигнал
<-sigChan
fmt.Println("\nПолучен сигнал остановки")

// 1. Сначала отменяем контекст (говорим горутинам завершаться)
cancel()

// 2. Даем 5 секунд на graceful shutdown
fmt.Println("Ожидаем завершения горутин (макс. 5 сек)...")

shutdownCtx, shutdownCancel := context.WithTimeout(context.Background(),
5*time.Second)
defer shutdownCancel()

// 3. Ждем либо завершения, либо таймута
select {
case <-shutdownCtx.Done():
    fmt.Println("Таймаут завершения истек, принудительно выходим")
case <-time.After(3 * time.Second): // имитируем ожидание завершения горутин
    fmt.Println("Все горутины завершены")
}

fmt.Println("Приложение остановлено")
}

func worker(ctx context.Context, name string) {
    for {
        select {
        case <-ctx.Done():
            fmt.Printf("%s: завершается (освобождаю ресурсы)...\n", name)
            time.Sleep(1 * time.Second) // имитируем освобождение ресурсов
            fmt.Printf("%s: полностью остановлен\n", name)
            return
        default:
            fmt.Printf("%s: работаю...\n", name)
            time.Sleep(300 * time.Millisecond)
        }
    }
}

```